

Predictive Health Monitoring of Engine Classification Using Machine Learning and NASA C-MAPSS Data

Kyle Keshmeshian

May 12, 2025

1 Introduction

The purpose of my project is to classify engine status with The NASA C-MAPSS dataset. By separating the engine data into two categories, “Normal” and “At Risk”, my goal was to use machine learning to accurately predict the operational status of each engine based on the RUL (Remaining Useful Life). I accomplished this by performing an exploratory data analysis, comprehensive feature engineering/selection, comparing a variety of machine learning models, and extensive hyperparameter tuning.

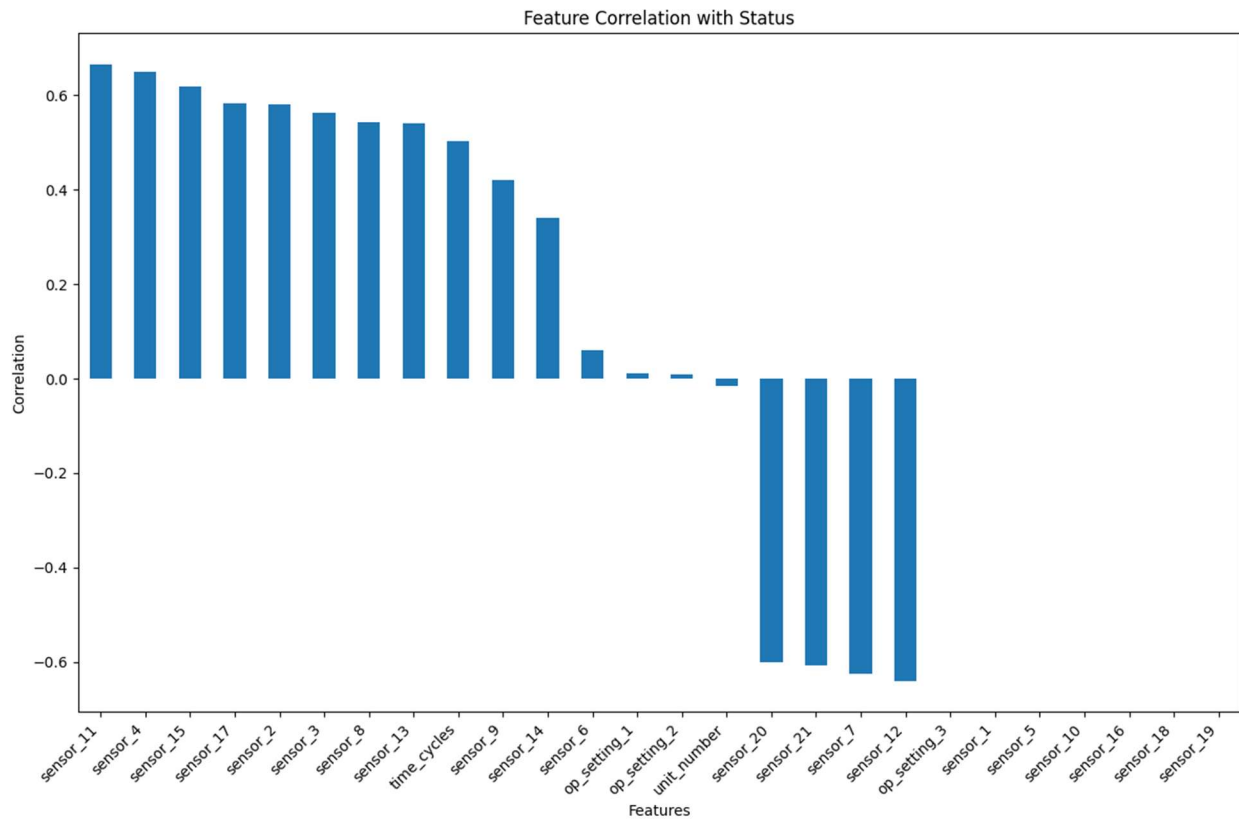
The NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) dataset simulated a series of flights within numerous conditions to monitor how NASA’s Turbofan Engine would degrade overtime.

2 Exploratory Data Analysis

2.1 Dataset Information

The NASA C-MAPSS dataset consists of 20,631 rows, 26 columns and no missing values. Every row contains data from a cycle simulated by one of NASA’s engines and the columns contain the engine ID (1-100), cycle (a single operational period), three operational settings, and 21 sensor measurements. Using these columns, I created the “status” column to use as the target for this project. If the engine is at risk of failure, then $RUL \leq 30$ and status equals 1. If the engine is operating normally then $RUL > 30$, and status equals 0. Of the 20,631 cycles, only 15% of those engines are considered “at risk” while the other 85% is operating normally.

2.2 Correlations & Feature Variance

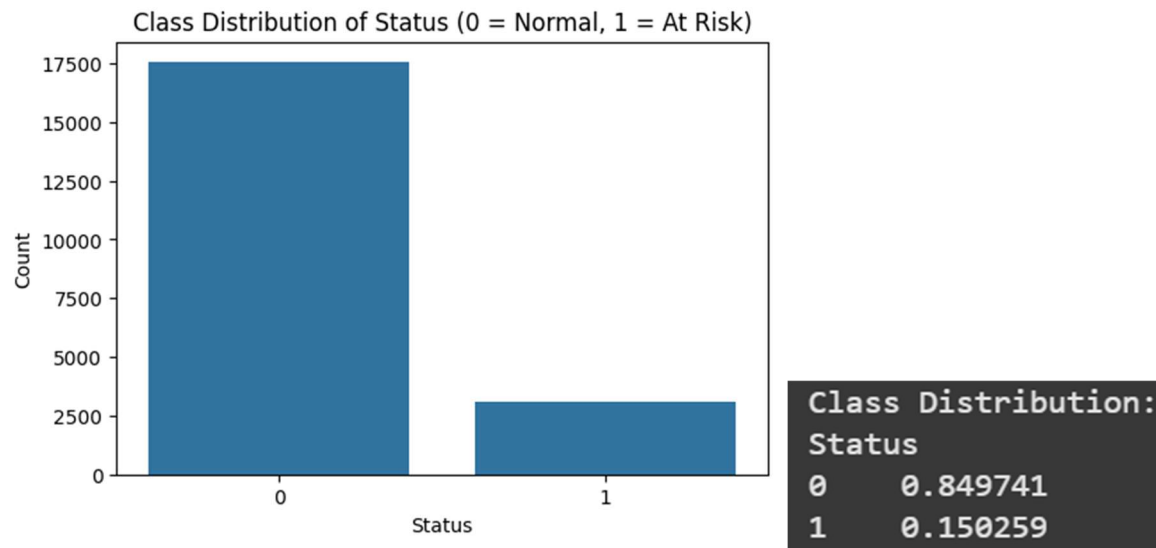


Top 10 Correlations with Status:		Bottom 10 Correlations with Status:	
Status	1.000000	sensor_21	-0.606480
sensor_11	0.665655	sensor_7	-0.625592
sensor_4	0.648406	sensor_12	-0.640174
sensor_15	0.618732	op_setting_3	NaN
sensor_17	0.583067	sensor_1	NaN
sensor_2	0.581404	sensor_5	NaN
sensor_3	0.561892	sensor_10	NaN
sensor_8	0.542910	sensor_16	NaN
sensor_13	0.539915	sensor_18	NaN
time_cycles	0.503679	sensor_19	NaN

Bottom 5 Feature Variances:	
op_setting_3	0.000000e+00
sensor_18	0.000000e+00
sensor_19	0.000000e+00
sensor_16	2.422479e-28
sensor_10	2.172333e-25

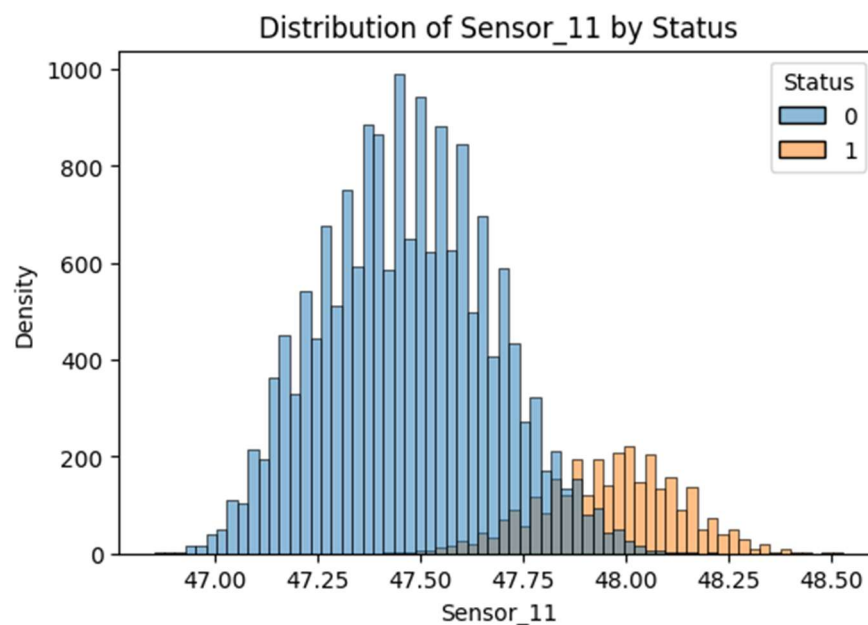
Before applying any machine learning models, I conducted a couple tests to determine which features to include in the model. We want to avoid a low correlation and a small variance because a low correlation suggests a weak linear relationship with the target and a small variance means that the feature remains constant throughout the experiment. By examining the Pearson correlation and the variance, we see that op_setting_3, sensor_1, sensor_5, sensor_10, sensor_16, sensor_18, and sensor_19 are irrelevant when determining engine status so to streamline the dataset, I removed those columns. On the other hand, we see that sensor_11 is very influential regarding engine status. After these changes, the dataset consists of 20 features.

2.3 Label Histogram & Class Imbalance

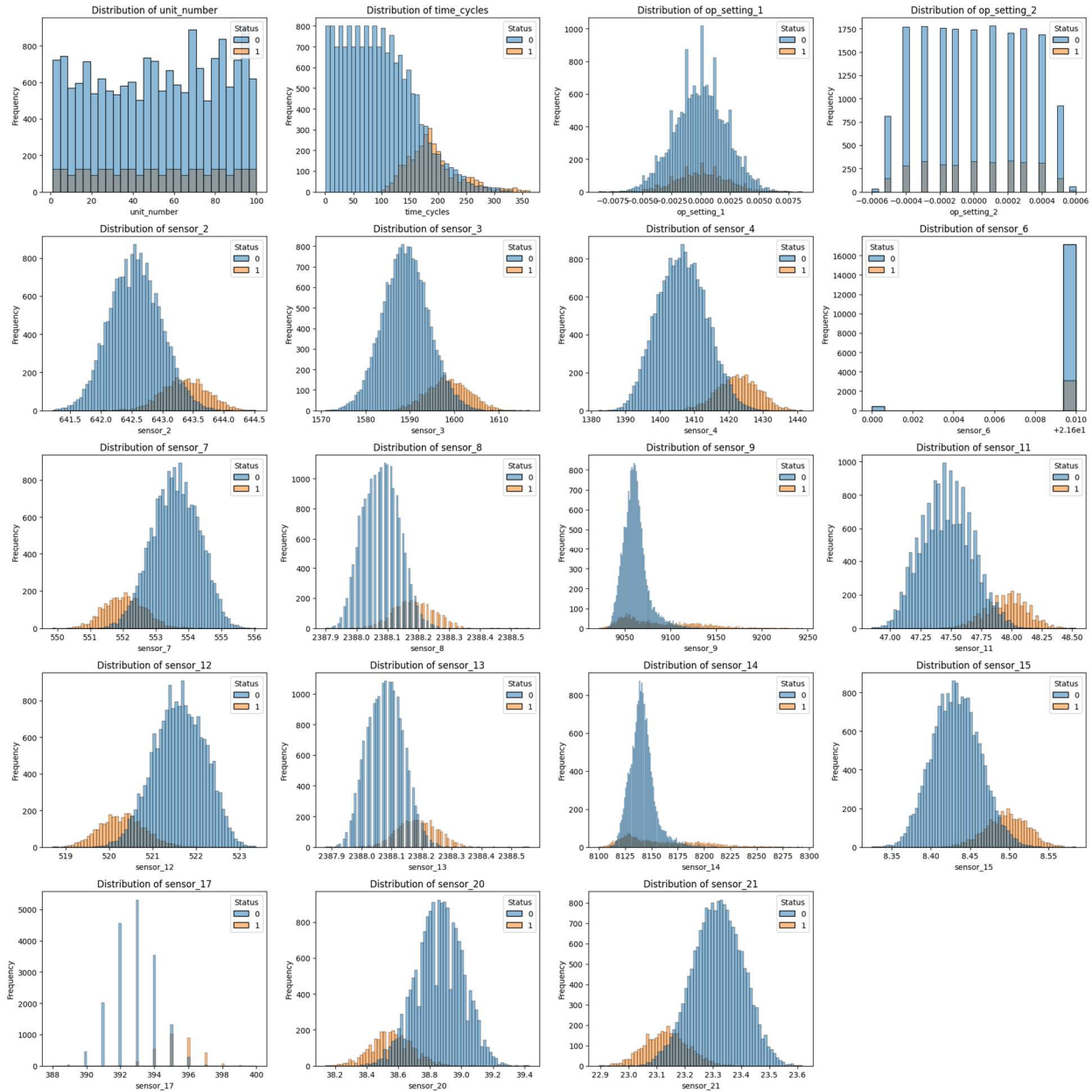


As depicted in the graph above, there is a major imbalance in the classes. 85% of the engines are under normal operating conditions (0) while the remaining 15% are at risk of failure (1). With my goal being classification, this could be troublesome without taking precautions. For example, later in the project when I appropriate evaluation metrics, I prioritized recall for this reason. I will go more into depth on that later, but another example is with `class_weight='balanced'` in Random Forest and stratify, these assist in maintaining class balance throughout the project. Biased models, in this context, means predicting that an engine is operating like normal when it may be at risk of failure could put lives in danger.

2.4 Feature Distributions & Standardization

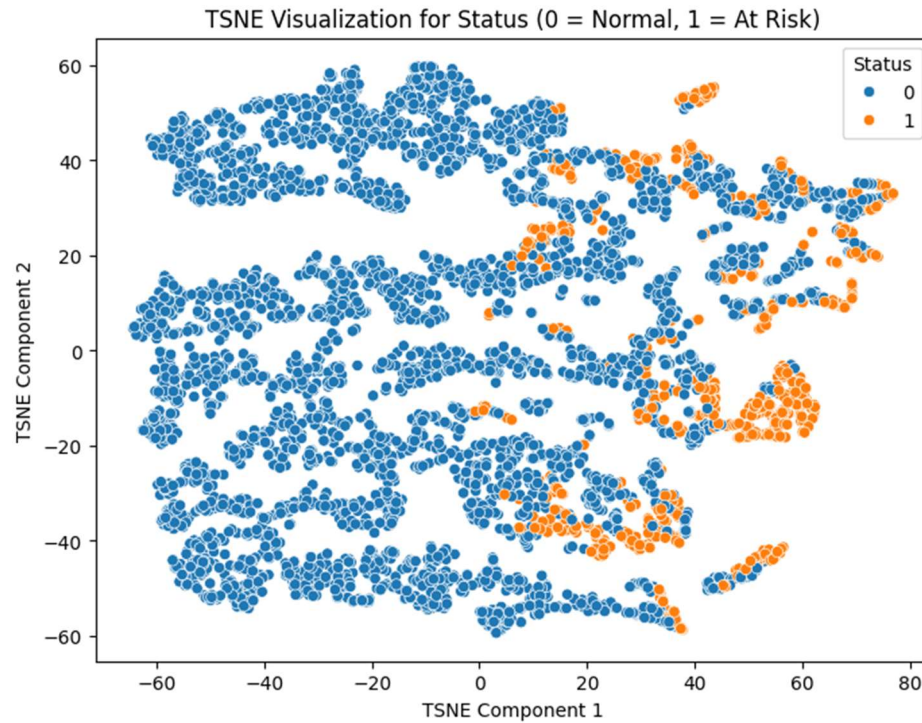


With sensor_11 being our feature with the highest correlation, I started here. In the context of this problem, this means that the value increased as the engine approached failure. The right skewness reflects the 85% normal and 15% class distribution that we see in the histogram above.



The varied shapes and skewedness displayed in the remaining 19 features heavily suggested that the data would benefit from standardization. After standardization means that every feature had their mean scaled down to zero and their variance to one. We scaled every feature so that we have a consistency when performing calculations and for everything to contribute equally to the models. This will prove to be very valuable when we consider models such as logistic regression, random forest, and more.

2.5 TSNE



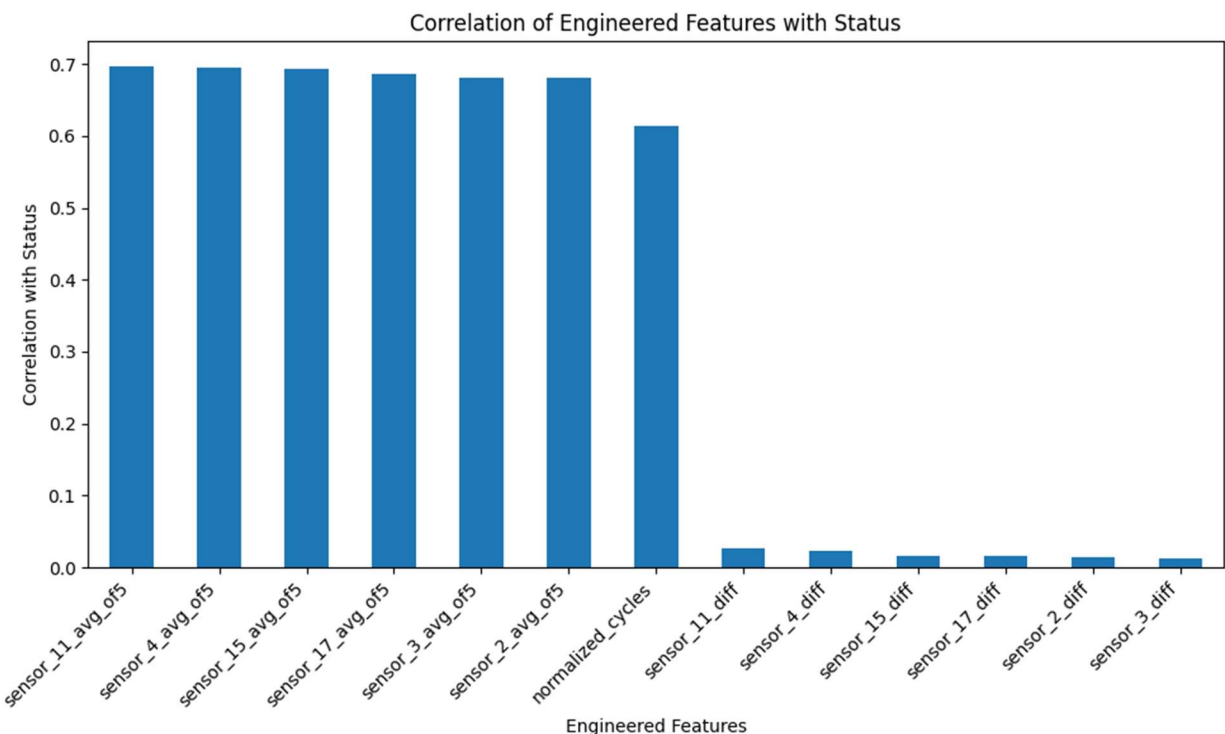
To check for separability, I applied T-distributed Stochastic Neighbor Embedding (t-SNE) to visualize the data in 2D. In this plot we see that although the unbalanced distribution of engine status is present, we do see relatively distinct clusters. As a result of the partial, it is reasonable to expect high AUC scores from our models. The clusters support my decision in which features to include in the data, but alongside those clusters is a overlap between the two classes. The overlap is to be expected due to the complex nature of the data, and it hints that my project may require a degree of feature engineering to ignore noise and achieve my goal.

3 Feature Engineering

3.1 Reasoning

To improve the performance of the machine learning models, I conducted comprehensive feature engineering and created an additional 13 features from our existing 20 with the goal of enhancing the accuracy of the predictions.

3.2 Successful Features



Sensor_11_avg_of5, along with the other 5 features with “_avg_of5” were created with the goal of smoothing out our data with the intention of assisting models such as XGBoosting and Random Forest. We achieve this by grouping each engine by their unit number and perform a rolling average 5 “cycles” at a time. This smooths out short-term fluctuations and highlights long-term trends. As we can see in the graph above, these features succeeded and achieved the highest correlation among engineered features because it better reflected the gradual degradation that leads to an engine potential failure.

Normalized_cycles was also very influential in determining the operating status of the engines because it was engineered to reflect the progression of the engine’s lifespan. Just as the name suggests, it was created by normalizing the cycle count for each engine by its maximum lifespan. A reason for its success and high correlation is because it greatly assisted in identifying patterns.

3.3 Unsuccessful Features

While I was able to engineer features that became crucial to my goal, not all features we're as helpful as expected. All the variables of the format, "sensor_#_diff", were not influential in determining the status of our engines. The idea behind the feature was to calculate the rate of change by the sensors and capture sudden changes in the data in order to link it to the failure of an engine but this was not the case. As shown in the graph above, sensor_#_diff were the least influential features, and it resulted in introducing noise and inconsistencies into the dataset.

3.4 Feature Selection

Top 15 Features by Mutual Information:		
	Feature	Mutual_Information
31	normalized_cycles	0.371528
19	sensor_11_avg_of5	0.303893
21	sensor_4_avg_of5	0.299128
23	sensor_15_avg_of5	0.294888
27	sensor_2_avg_of5	0.284267
25	sensor_17_avg_of5	0.282936
29	sensor_3_avg_of5	0.274538
11	sensor_11	0.265888
6	sensor_4	0.250846
12	sensor_12	0.244935
8	sensor_7	0.233073
15	sensor_15	0.224943
18	sensor_21	0.217842
17	sensor_20	0.209696
4	sensor_2	0.196315

After engineering features, I calculated the mutual information scores to identify the most predictive subset in the data. To avoid overfitting, I found that it was best to include the top 15 out of 33 features moving forward. These 15 include normalized_cycles, sensor_11_avg_of5, sensor_4_avg_of5, sensor_15_avg_of5, sensor_2_avg_of5, sensor_17_avg_of5, sensor_3_avg_of5, sensor_11, sensor_4, sensor_12, sensor_7, sensor_15, sensor_21, sensor_20, and sensor_2. This revealed that the most predictive features are the ones that capture trends.

4 Application of ML Models

4.1 Overview

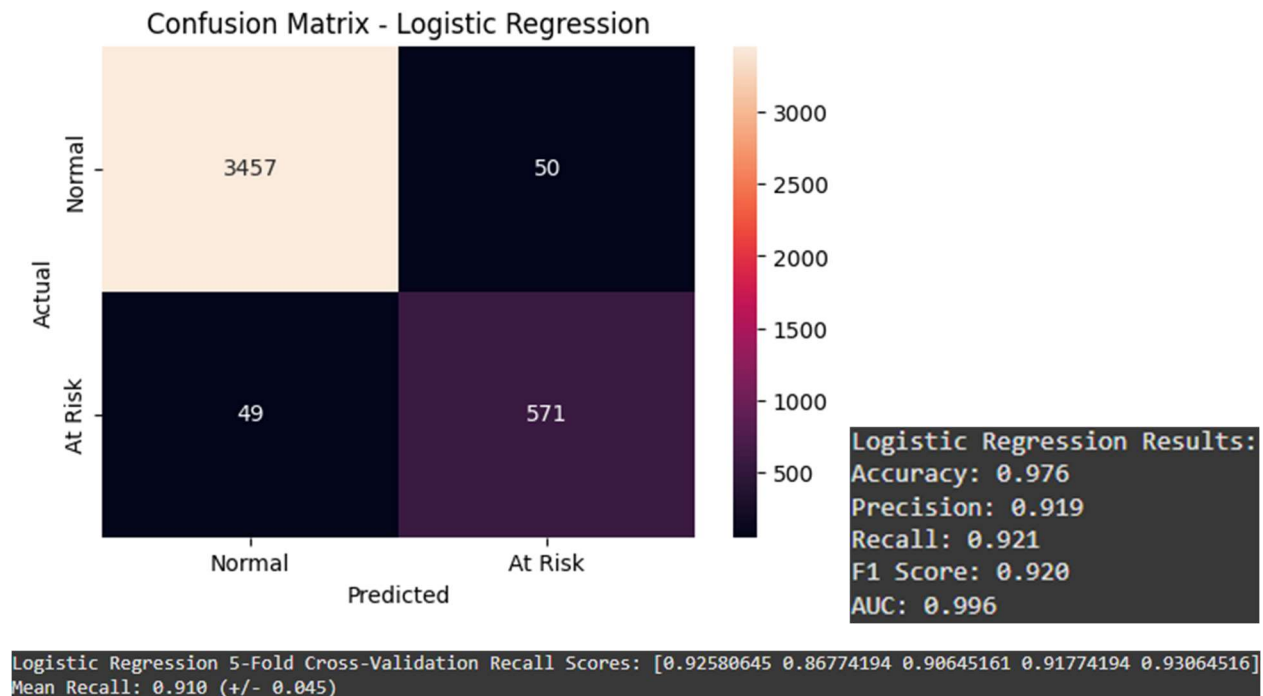
There are many machine learning models that can classify the dataset, but I chose to use Logistic Regression, XGBoost, Random Forests, and Multi-Layer Perceptrons (MLP). Using an 80-20 train-test split, the testing set contained 4,127 samples and each model was run with a focus on maximizing Recall as our metric to track performance. Included in this application are hyperparameter tuning using grid search, 5 fold cross validation, and visualizations such as confusion matrices and ROC curves. In every model I incorporated `class_weight='balanced'` since 85% of the data consists of engines that are operating normally.

I found that recall is the most important because it measures the proportion of engines that are at risk of failure that are correctly classified by the model. Due to the context of this real-world application, I placed an emphasis on the false negatives displayed in the confusion matrices because a false negative means that an engine that is at risk of failure was incorrectly classified.

Understanding the confusion matrices within this context is also crucial in determining the best model. True negatives are normal engines correctly classified, true positives are at risk engines correctly classified, false negatives are at risk engines incorrectly classified, and false positives are normal engines incorrectly classified.

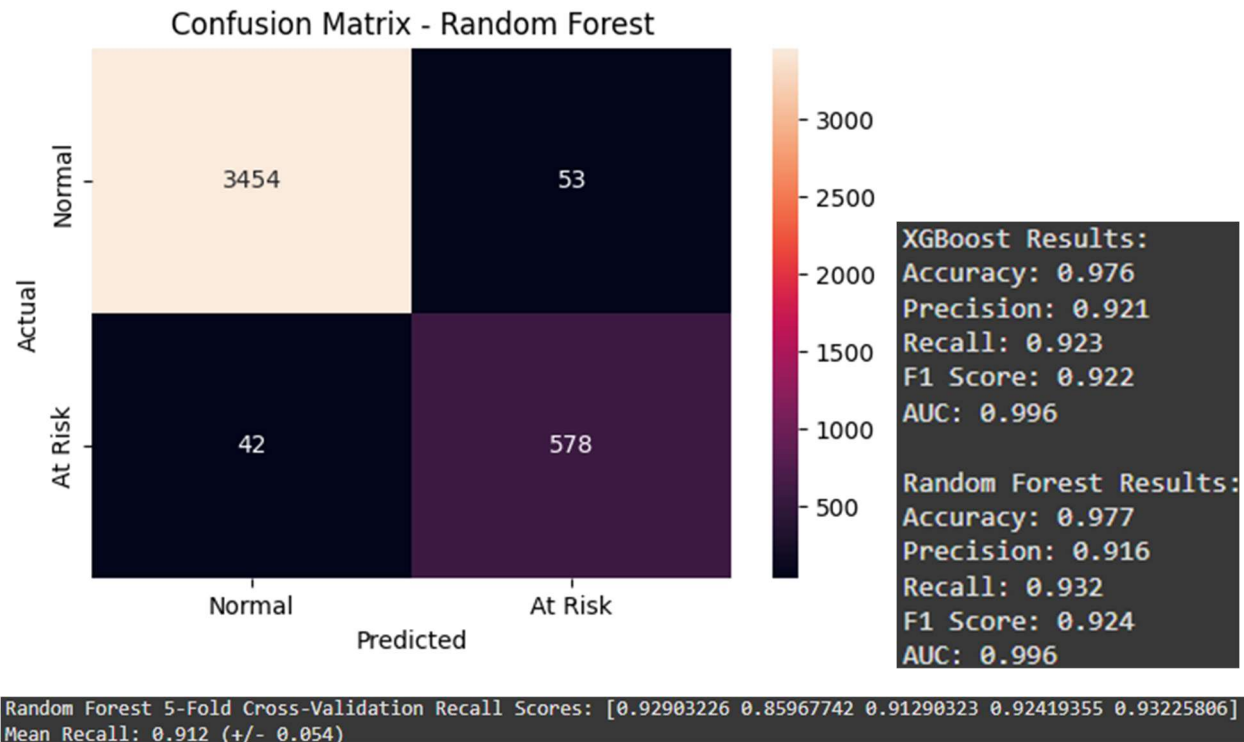
As for the remaining metrics, accuracy measures the proportion of correct predictions (for both statuses), precision measures the proportion of predicted at-risk engines that are correctly identified, F1 is the mean of precision and recall, and AUC measures the model's ability to distinguish between normal and at risk statuses using the ROC curve.

4.2 Linear Models (Logistic Regression)



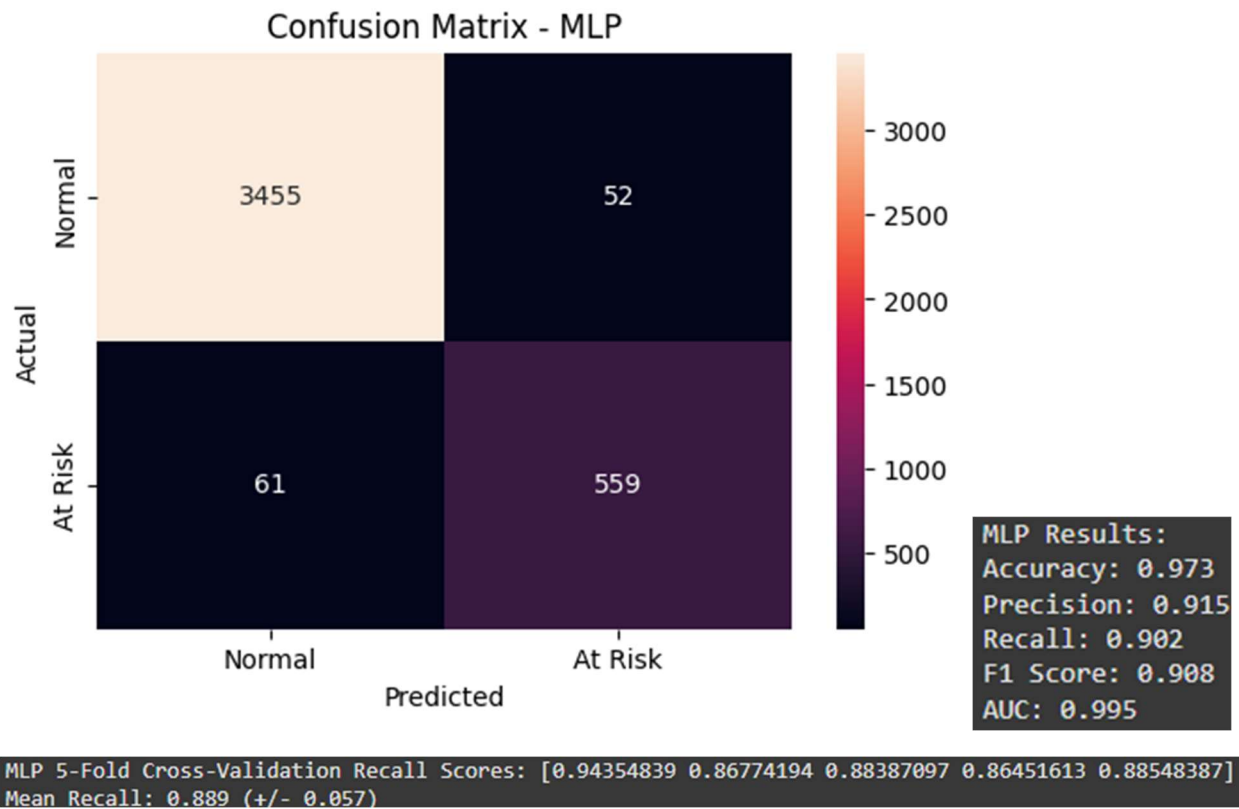
Starting with Linear Models, I performed Logistic Regression using scikit learn. It achieved an Accuracy of 0.976, Precision of 0.919, Recall of 0.921, F1 of 0.920, and AUC of 0.996. Interpreting the confusion matrix, we see above that logistic regression resulted in 49 false negatives, meaning that 7.9% of at risk engines were classified incorrectly. The 5 Fold cross validation resulting in a mean recall of 0.920 ± 0.045 shows the consistency of logistic regression's ability to classify. Overall logistic regression provided a great baseline to go from, but linear models in general are limited by their simplicity and inability to capture complex feature relationships.

4.3 Tree Based Models (XGBoost & Random Forest)



With Based Models, both XGBoost and Random Forest could be applied. Beginning with XGBoost, it achieved an Accuracy of 0.976, Precision of 0.921, Recall of 0.923, F1 Score of 0.922, and AUC of 0.996. And as for random forest, it achieved an Accuracy of 0.977, Precision of 0.916, Recall of 0.932, F1 Score of 0.924, and AUC of 0.996. Although it was close, random forest performed better overall, so I turned my attention to that. Interpreting the confusion matrix, we see it resulted in 42 false negatives, meaning that 6.8% of at-risk engines were classified incorrectly, this is an improvement from logistic regression. The 5 Fold cross validation resulted in a mean recall of 0.912 ± 0.054 . We can see that the standard deviation is higher than logistic regression's, this could be caused by the randomness of random forests or the variation along folds.

4.4 Neural Networks (MLPs)



Moving to neural network models, using a Multi-Layer Perceptron (MLP) configured with two hidden layers (100 and 50 neurons), and ReLU activation. It achieved an Accuracy of 0.973, Precision of 0.915, Recall of 0.902, F1 Score of 0.908, and AUC of 0.995. Interpreting the confusion matrix, we see it resulted in 61 false negatives, meaning that 10% of at-risk engines were classified incorrectly, this is a downgrade from logistic regression. The 5 Fold cross validation resulted in a mean recall of 0.889 ± 0.057 . This suggests that MLPs, in this case, are less consistent in classification and have more variability among folds.

4.5 Hyperparameter Tuning

```
Best Parameters: {'class_weight': 'balanced', 'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 100}
```

```
Tuned Random Forest Results:
```

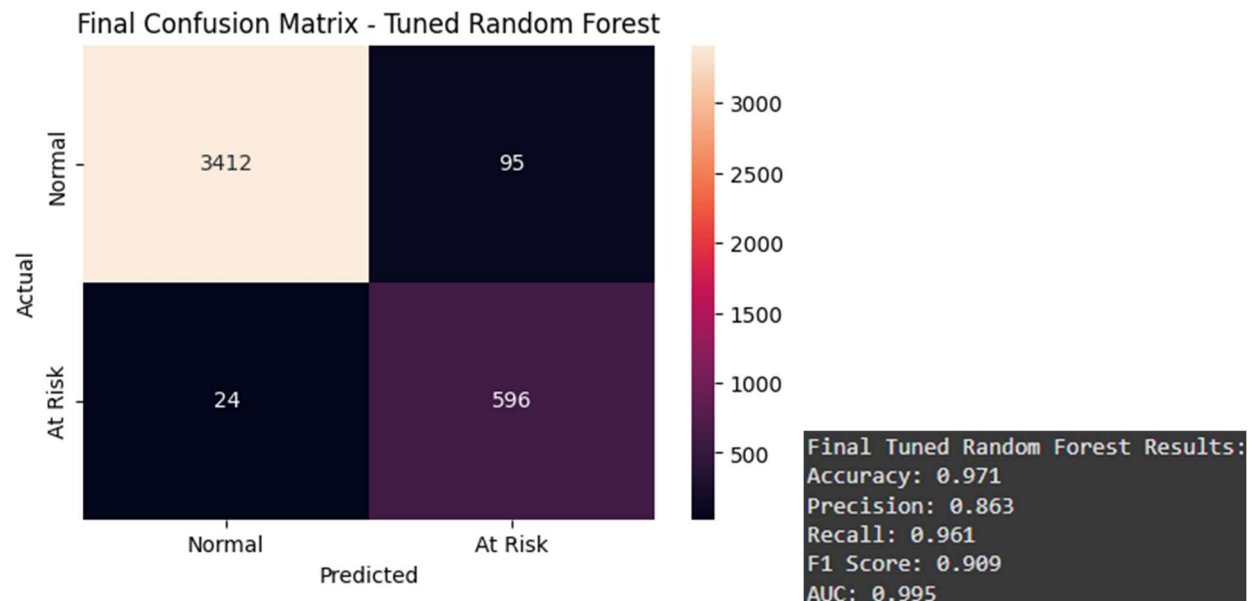
```
Accuracy: 0.971
```

```
Precision: 0.865
```

```
Recall: 0.960
```

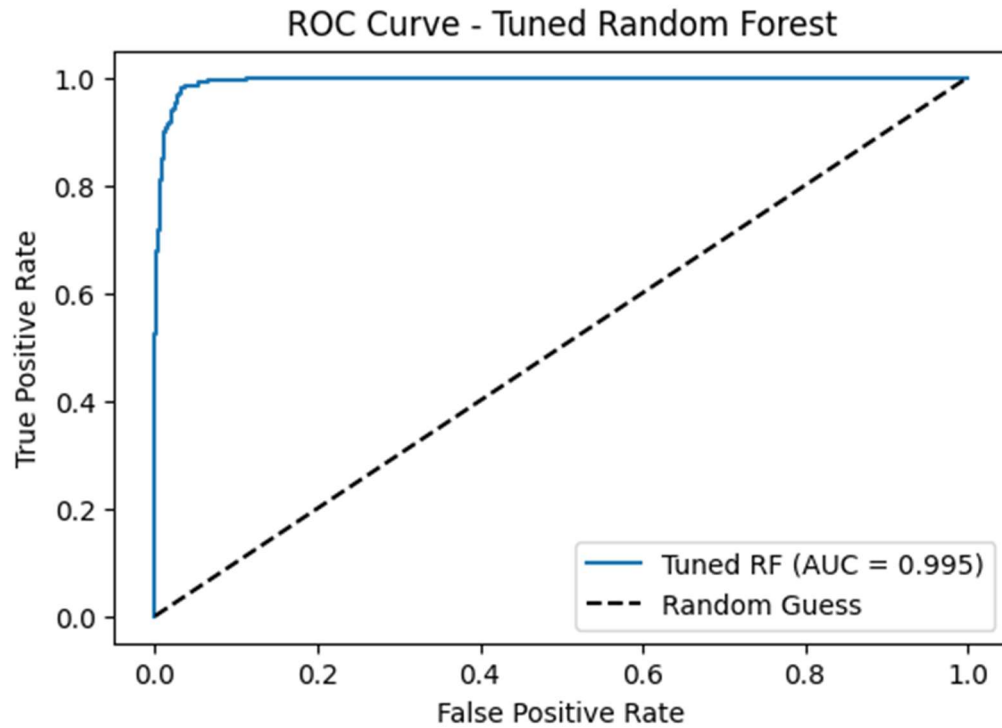
```
F1 Score: 0.910
```

```
AUC: 0.995
```



```
Final Tuned Random Forest 5-Fold Cross-Validation Recall Scores: [0.95      0.93548387 0.95483871 0.9483871  0.95      ]  
Mean Recall: 0.948 (+/- 0.013)
```

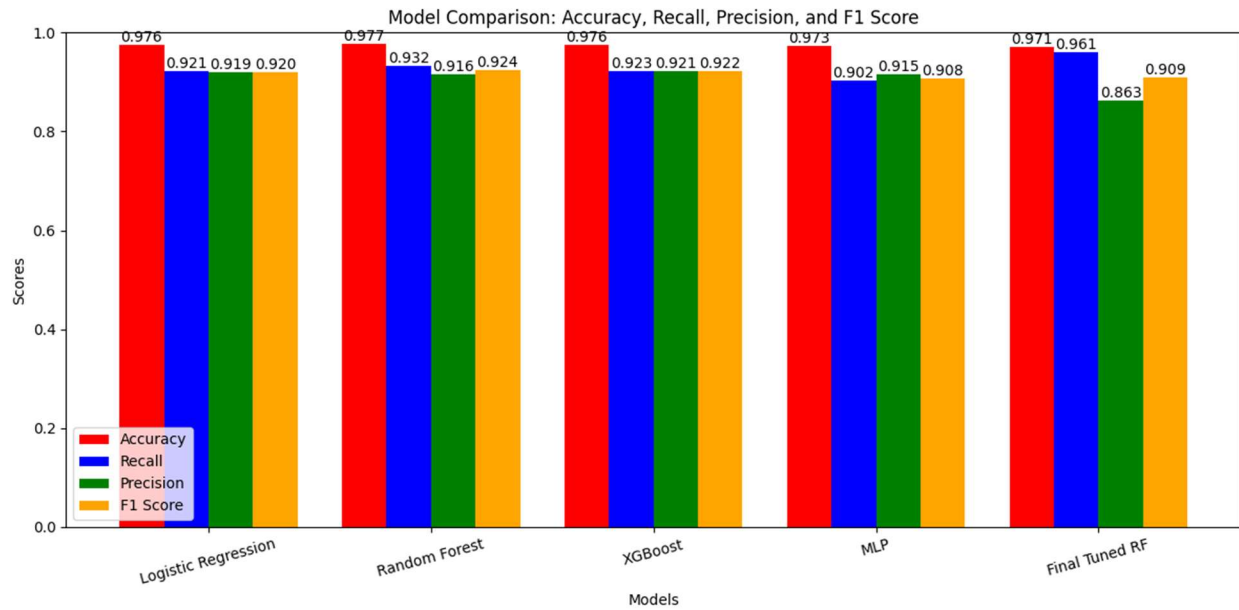
Across all models, it was clear that the random forest implementation provided the best and safest results for the classification of engine status. Moving forward with random forest, we focused on hyperparameter tuning to optimize the recall. I performed this with a grid search with cross validation to find the optimal number of trees (200), tree depth (10), and the number of splits (5). This model achieved an Accuracy of 0.971, Precision of 0.863, Recall of 0.961, F1 Score of 0.909, and AUC of 0.995. Interpreting the confusion matrix, we see it resulted in 24 false negatives, meaning that only 3.9% of at-risk engines were classified incorrectly. The 5 Fold cross validation resulting in a mean recall of 0.948 ± 0.013 is the highest of all models and has the lowest variability among folds.



The ROC curve for the tuned random forest model visualizes the model's ability to almost perfect classify the operating status of the engines in our dataset every time. This model is backed by a very high AUC of 0.995 and the steepness at the beginning of the graph suggests that the model has a very high recall. These results consistently blow all other models out of the water and its success validates the approach used throughout the project.

5 Conclusion

5.1 Final Model



This project successfully developed a predictive health monitoring system for aircraft engines using the NASA C-MAPSS dataset by achieving the goal of classifying engine status (normal vs. at risk) with high reliability. Through comprehensive feature engineering, we derived critical features like `normalized_cycles` and `sensor_11_avg_of5`, which enhanced model performance by capturing degradation trends. Feature selection using mutual information narrowed down the dataset to 15 key features. Comparative analysis of Logistic Regression, XGBoost, Random Forest, and MLP highlighted Random Forest as our candidate for hyperparameter tuning. The Final Tuned Random Forest, after being optimized with grid search, achieved a recall of 0.961, missing only 24 at-risk engines (3.9%), with an AUC of 0.995, demonstrating its strong classification capabilities. With the ROC curve confirming its effectiveness this solution minimizes safety risks and established a robust model for NASA's Turbofan Engine.

5.2 Limitations & Future

The most prominent limitation was with the NASA C-MAPSS dataset. Without knowing what the sensor data is measuring, I was unable to interpret and translate the data into our real-world situation. Another constant hurdle throughout the project was the class imbalance within the data. Of all 20,631 samples, only 15% fit the criteria to be at risk of failure.

In the future I'd like to see sensor specific data and see how my model performs against different types of engines in real time.

5.3 Key Takeaways

My biggest takeaway came from seeing the massive success of my engineered features. Normalizing the time cycles and smoothing the sensor data by computing the average effectively captured the trends in how engines degraded overtime and I believe that my model would not classify nearly as good as it does without it. My second biggest takeaway was the importance of choosing the appropriate metric to evaluate. Typically, we are expected to chase accuracy but, in this case, where we prioritize safety and want to mitigate risks, it was a no brainer to make my decisions based on recall.

6 Sources

Behrad3d. (n.d.). NASA C-MAPSS aircraft engine simulator data [Data set]. Kaggle.
<https://www.kaggle.com/datasets/behrad3d/nasa-cmaps/data>

NASA. (n.d.). C-MAPSS aircraft engine simulator data [Data set]. NASA Data Portal.
<https://data.nasa.gov/dataset/c-mapss-aircraft-engine-simulator-data>